

Sistema de Tráfico urbano (Grafos + Tablas hash)

Paola Sarahi Zermeño Ojeda, Vladimir Suárez Vega, Diana Valeria Zermeño Ojeda, Diego Sebastián Medina Lara

Universidad Autónoma de Aguascalientes, Sistemas Computacionales, Grupo 3-B, Aguascalientes, México

al551467@edu.uaa.mx
al548562@edu.uaa.mx
al518308@edu.uaa.mx
al283608@edu.uaa.mx

Resumen—Este trabajo presenta la implementación en C++ de un sistema para la gestión de una red vial modelada como un grafo dinámico dirigido con pesos. El grafo permite la inserción y eliminación de nodos y aristas en tiempo de ejecución, soportando algoritmos clásicos como Dijkstra, BFS y DFS, todos adaptados para manejar nodos eliminados. Se integra una tabla hash con encadenamiento para el registro y búsqueda eficiente de vehículos, logrando operaciones en tiempo promedio $O(1)$. La interfaz de usuario, basada en menús interactivos, facilita la creación, visualización y análisis de la red. El diseño prioriza la eficiencia y cumplimiento de restricciones académicas, evitando el uso de estructuras STL en componentes críticos como el min-heap y BFS. Este sistema es una base sólida para aplicaciones de simulación de tráfico y gestión de redes.

Palabras clave—grafo dirigido, estructuras dinámicas, Dijkstra, min-heap, BFS, DFS, tabla hash, C++

I. INTRODUCCIÓN

El presente trabajo desarrolla un sistema de gestión de redes viales utilizando estructuras de datos. El núcleo del sistema es un **grafo dirigido con pesos** que modela intersecciones (nodos) y tramos viales (aristas). A diferencia de implementaciones estáticas comunes, este grafo es completamente dinámico: permite crear y eliminar nodos en cualquier momento sin reconstruir la estructura.

Además, se incorpora una tabla hash para el registro y búsqueda eficiente de vehículos que circulan por la red, permitiendo operaciones de alta, baja y consulta.

Los objetivos específicos:

- Implementar un grafo dinámico con lista de adyacencia y matriz.
- Desarrollar un min-heap propio para el algoritmo de Dijkstra.
- Adaptar BFS y DFS para trabajar con nodos.
- Diseñar tablas hash.
- Implementar una interfaz de usuario clara y funcional.

II. METODOLOGÍA

La metodología empleada en el desarrollo del sistema se basó en un enfoque modular orientado a estructuras de datos

dinámicas y persistencia de información mediante archivos externos. Con el objetivo de construir una aplicación capaz de gestionar redes de nodos, realizar recorridos sobre un grafo y administrar un registro de vehículos, se definieron cuatro componentes principales: gestión de grafos, tablas hash, procesamiento de archivos CSV, y controladores de persistencia mediante archivos binarios. Cada componente se diseñó con independencia funcional, permitiendo su integración gradual en el programa principal.

2.1 Diseño de la Arquitectura General

El sistema se estructuró bajo un modelo modular donde cada archivo .h o .cpp cumple un rol específico:

- **Módulo de Grafo:** manejo dinámico de nodos y aristas, recorridos BFS, DFS y cálculo de rutas mínimas mediante Dijkstra.
- **Módulo HashRed y HashVehículos:** almacenamiento eficiente y acceso rápido a información mediante funciones hash y encadenamiento.
- **Módulo de manejo de CSV:** lectura y escritura de redes completas desde archivos externos.
- **Módulo de persistencia binaria:** control de versiones para archivos de red y vehículos mediante contadores .dat.
- **Main:** interfaz del usuario, menús y coordinación general.

El diseño modular permite que cualquier componente pueda extenderse sin afectar a los demás, manteniendo el principio de bajo acoplamiento.

2.2 Gestión del Grafo

Se implementó un grafo **dinámico**, representado internamente mediante:

- Listas enlazadas para almacenar nodos y aristas.
- Una matriz de adyacencia para visualización.
- Una lista de adyacencia para operaciones de recorrido.

La estructura admite **altas y bajas en tiempo de ejecución**, lo cual permite simular redes variables. Los métodos implementados incluyen:

- Inserción y eliminación de nodos.
- Inserción y eliminación de aristas con peso.
- Detección de existencia de nodos.

- Impresión en matriz y lista.
- Recorridos: **BFS**, **DFS**.
- Obtención de ruta mínima mediante **Dijkstra**, con medición de tiempo de ejecución.

Estas características permitieron cubrir los aspectos solicitados.

2.3 Tablas Hash para Nodos y Vehículos

Se utilizaron dos tablas hash independientes:

- **HashRed**, asociada a la red de nodos cargada desde CSV.
- **HashVehículos**, utilizada para registrar información de vehículos.

Ambas se implementaron con:

- Un arreglo estático de tamaño fijo (10 posiciones).
- Listas enlazadas para manejo de colisiones (encadenamiento).
- Operaciones de alta, búsqueda, baja y recorrido.

Esta estructura permitió la integración con los módulos de grafo para las operaciones solicitadas.

2.4 Archivos CSV

Para la transferencia de información entre sesiones, se empleó un módulo encargado de:

- leer archivos CSV que contienen nodos y aristas,
- reconstruir la red cargando primero nodos y después aristas,
- distribuir operaciones de alta/baja hacia archivos existentes.

El sistema utiliza un mecanismo de **"guardar" o "guardar como"**, permitiendo conservar versiones de la red. Cada archivo CSV contiene:

- lista de nodos con ID y nombre,
- lista de aristas con nodo origen, nodo destino y peso.

Este módulo proporciona el puente con la estructura interna del grafo.

2.5 Control de Versiones mediante Archivos Binarios

La metodología incluyó el uso de archivos binarios como contadores persistentes para mantener integridad en el número de archivos generados:

- **ContRed.dat** para archivos de red
- **ContVehiculos.dat** para archivos de vehículos

Cada archivo mantiene un contador que se actualiza con cada nueva operación de creación. Este mecanismo aporta:

- Control automático de nombres (red.csv, red2.csv, ...).
- Seguimiento de nuevas versiones.
- Persistencia entre ejecuciones.

Además, se implementó verificación de existencia, lectura segura y actualización secuencial, lo cual permitió garantizar estabilidad en ejecución.

2.6 Interfaz del Usuario y Flujo de Operación

El programa principal se diseñó sobre un menú interactivo que encapsula todas las funcionalidades del sistema. El flujo

sigue una estructura jerárquica:

- Opciones de Red y Hash (carga, creación, altas, bajas).
- Visualización del Grafo en dos formas.
- Recorridos y cálculo de rutas óptimas.
- Gestión de Vehículos.
- Salida.

Esta aproximación asegura que el usuario interactúe con el sistema de manera clara y ordenada, preservando la modularidad conceptual presentada en la arquitectura.

III. DESARROLLO Y RESULTADOS

El desarrollo del sistema se llevó a cabo mediante la implementación de diversos módulos funcionales que interactúan entre sí para permitir la construcción dinámica de redes, la ejecución de algoritmos de grafos, la gestión de vehículos y la persistencia de datos mediante archivos CSV y archivos binarios. A continuación, se describen los principales componentes y las decisiones técnicas adoptadas durante la programación.

3.1 Implementación del Grafo Dinámico

El módulo central del sistema es la clase **Grafo**, encargada de representar y manipular una red de nodos interconectados. Se desarrolló utilizando dos estructuras internas:

- Lista de adyacencia, para operaciones de recorrido y manipulación dinámica.
- Matriz de adyacencia, destinada exclusivamente a la visualización estructural.

Resultados

La lista de adyacencia y matriz de adyacencia se muestran en las figuras 1 y 2, respectivamente.

```
===== Matriz y Lista del Grafo <=====
[1] Mostrar Matriz.
[2] Mostrar Lista de Adyacencia.
[0] Volver al menú anterior.

Elige una opción: 2

      Lista de adyacencia
0 [Avenida_Adolfo_Lopez] : (1, peso=1.3) (6, peso=1.8)
1 [Glorieta_BenitoJuarez] : (2, peso=6.3)
2 [UAA] : (4, peso=3.2)
3 [Altaria] : (2, peso=4)
4 [Costco] : (5, peso=7.5) (3, peso=2.2) (1, peso=5.5)
5 [Seguridad_Vial] : (6, peso=5.8)
6 [Isla_SanMarcos] : (5, peso=5.7) (2, peso=8.5)
7 [] :
8 [] :
9 [] :
10 [] :
11 [] :
```

Figura 1. Lista de adyacencia en ejecución.

```
===== Matriz y Lista del Grafo <=====
[1] Mostrar Matriz.
[2] Mostrar Lista de Adyacencia.
[0] Volver al menú anterior.

Elige una opción: 1

      Matriz de adyacencia:
      0      1      2      3      4      5      6      7      8      9      10      11
0 | 0      1.3  INF  INF  INF  INF  1.8  INF  INF  INF  INF  INF
1 | INF  0      6.3  INF  INF  INF  INF  INF  INF  INF  INF  INF
2 | INF  INF  0      INF  3.2  INF  INF  INF  INF  INF  INF  INF
3 | INF  INF  4      0      INF  INF  INF  INF  INF  INF  INF  INF
4 | INF  5.5  INF  2.2  0      7.5  INF  INF  INF  INF  INF  INF
5 | INF  INF  INF  INF  INF  0      5.8  INF  INF  INF  INF  INF
6 | INF  INF  INF  8.5  INF  INF  5.7  0      INF  INF  INF  INF
7 | INF  INF  INF  INF  INF  INF  INF  0      INF  INF  INF  INF
8 | INF  INF  INF  INF  INF  INF  INF  INF  0      INF  INF  INF
9 | INF  INF  INF  INF  INF  INF  INF  INF  INF  0      INF  INF
10 | INF  INF  INF  INF  INF  INF  INF  INF  INF  INF  0      INF
11 | INF  INF  INF  INF  INF  INF  INF  INF  INF  INF  INF  0
```

Figura 2. Matriz de adyacencia en ejecución.

1) Alta y baja de nodos (desde archivos)

El grafo soporta operaciones de actualización en tiempo de ejecución.

Para agregar un nodo se ejecuta:

- creación de un objeto nodo,
- asignación de su identificador y nombre,
- inserción dentro de la estructura principal,
- inicialización de su lista de aristas.

Para la baja, el sistema:

- elimina nodos existentes,
- recorre el grafo para eliminar aristas que apunten a dicho nodo.
- actualiza la matriz y listas asociadas.

Este diseño proporciona un grafo completamente mutable.

Resultados

La ejecución de altas y bajas de nodos se muestra en las figuras 3 y 4, respectivamente.

```
====> Nodos y Aristas (Manejo desde Archivos) <====
[1] Crear nueva Red.
[2] Alta de Nodos.
[3] Alta de Aristas.
[4] Baja de Nodos.
[5] Baja de Aristas.
[0] Volver al menú anterior.

Elige una opción: 2

Archivos de Redes de Nodos:

[1] red.csv

En qué archivo desea agregar: 1

[1] Modificar archivo y guardar.
[2] Modificar archivo y guardar como uno nuevo.

Escoja una opción para guardar las modificaciones: 2

Los nodos actuales en el archivo son:
Id: 0 | Nombre: Avenida_Adolfo_Lopez
Id: 1 | Nombre: Glorieta_BenitoJuarez
Id: 2 | Nombre: UAA
Id: 3 | Nombre: Altaria
Id: 4 | Nombre: Costco
Id: 5 | Nombre: Seguridad_Vial
Id: 6 | Nombre: Isla_SanMarcos

Cuántos nodos deseas agregar: 1

Dame el nombre del nodo 7: Plaza_Patria

El archivo fue guardado correctamente.
```

Figura 3. Alta de nodo en ejecución.

```
====> Nodos y Aristas (Manejo desde Archivos) <====
[1] Crear nueva Red.
[2] Alta de Nodos.
[3] Alta de Aristas.
[4] Baja de Nodos.
[5] Baja de Aristas.
[0] Volver al menú anterior.

Elige una opción: 4

Archivos de Redes de Nodos:

[1] red.csv
[2] red2.csv
[3] red3.csv
[4] red4.csv
[5] red5.csv

Qué archivo desea eliminar: 5

[1] Modificar archivo y guardar.
[2] Modificar archivo y guardar como uno nuevo.

Escoja una opción para guardar las modificaciones: 1

Los nodos actuales en el archivo son:
Id: 0 | Nombre: Avenida_Adolfo_Lopez
Id: 1 | Nombre: Glorieta_BenitoJuarez
Id: 2 | Nombre: UAA
Id: 3 | Nombre: Altaria
Id: 4 | Nombre: Costco
Id: 5 | Nombre: Seguridad_Vial
Id: 6 | Nombre: Isla_SanMarcos
Escribe el ID del nodo que deseas eliminar: 4

El nodo con el id 4 y sus aristas adyacentes han sido eliminadas.
```

Figura 4. Eliminación de nodo en ejecución.

2) Alta y baja de aristas (desde archivos)

Las aristas se agregan mediante un método que:

- recibe origen, destino y peso,
- crea un nodo de arista,
- enlaza dinámicamente la estructura,
- actualiza la matriz de adyacencia.

La eliminación opera de manera similar recorriendo las listas enlazadas de cada nodo.

Resultados

La ejecución de altas y bajas de nodos se muestra en las figuras 5, 6 y 6.1, respectivamente.

```
====> Nodos y Aristas (Manejo desde Archivos) <====
[1] Crear nueva Red.
[2] Alta de Nodos.
[3] Alta de Aristas.
[4] Baja de Nodos.
[5] Baja de Aristas.
[0] Volver al menú anterior.

Elige una opción: 3

Archivos de Redes de Nodos:

[1] red.csv
[2] red2.csv
[3] red3.csv
[4] red4.csv

Qué archivo desea agregar: 1

[1] Modificar archivo y guardar.
[2] Modificar archivo y guardar como uno nuevo.

Escoja una opción para guardar las modificaciones: 2

Nodo origen: 0 (Avenida_Adolfo_Lopez)

Deseas agregar una arista desde Avenida_Adolfo_Lopez hacia Glorieta_BenitoJuarez? [S/N]: N
Deseas agregar una arista desde Avenida_Adolfo_Lopez hacia UAA? [S/N]: S
Peso: 2.1
Deseas agregar una arista desde Avenida_Adolfo_Lopez hacia Altaria? [S/N]: N
Deseas agregar una arista desde Avenida_Adolfo_Lopez hacia Costco? [S/N]:
```

Figura 5. Alta de arista en ejecución.

```
====> Nodos y Aristas (Manejo desde Archivos) <====
[1] Crear nueva Red.
[2] Alta de Nodos.
[3] Alta de Aristas.
[4] Baja de Nodos.
[5] Baja de Aristas.
[0] Volver al menú anterior.

Elige una opción: 5

Archivos de Redes de Nodos:

[1] red.csv
[2] red2.csv

En qué archivo desea eliminar: 2

[1] Modificar archivo y guardar.
[2] Modificar archivo y guardar como uno nuevo.

Escoja una opción para guardar las modificaciones: 1
```

Figura 6. Primera parte de baja arista.

```
Escoja una opción para guardar las modificaciones: 1

Las aristas actuales en el archivo son:
Origen: 0 | Destino: 6 | Peso: 1.8
Origen: 0 | Destino: 1 | Peso: 1.3
Origen: 1 | Destino: 2 | Peso: 6.3
Origen: 2 | Destino: 4 | Peso: 3.2
Origen: 3 | Destino: 2 | Peso: 4
Origen: 4 | Destino: 1 | Peso: 5.5
Origen: 4 | Destino: 3 | Peso: 2.2
Origen: 4 | Destino: 5 | Peso: 7.5
Origen: 5 | Destino: 6 | Peso: 5.8
Origen: 6 | Destino: 2 | Peso: 8.5
Origen: 6 | Destino: 5 | Peso: 5.7
Origen: 0 | Destino: 7 | Peso: 3.5
Origen: 7 | Destino: 4 | Peso: 4.5
Escriba el origen de la arista que desea eliminar: 7

[1] Origen: 7 | Destino: 4 | Peso: 4.5

Elija la arista que desea eliminar: 1

La arista con:
Origen: 7 | Destino: 4 | Peso: 4

Ha sido eliminada.
Desea eliminar otra arista? [S/N]: n

El archivo fue guardado correctamente.
```

Figura 6.1. Segunda parte de baja arista.

A demás se agrega el menú de operaciones que funcionan de manera similar desde grafo, como se muestra en la figura 7.

```
=====
====> Nodos y Aristas (Manejo desde Grafo) <====
[1] Alta de Nodos.
[2] Alta de Aristas.
[3] Baja de Nodos.
[4] Baja de Aristas.
[0] Volver al menú anterior.

Elige una opción:
```

Figura 7. Operaciones que se pueden realizar desde grafo.

3.2 Implementación de Recorridos

El sistema integra tres algoritmos clásicos:

1) Breadth-First Search (BFS)

Realizado mediante:

- Una cola FIFO.
- Un arreglo de visitados.
- Un recorrido secuencial sobre la lista de adyacencia.

Permite obtener un recorrido por niveles desde un nodo inicial.

Resultados

A continuación, en la figura 8 se muestra el algoritmo BFS en ejecución.

```
=====
====> Recorridos del Grafo <====
[1] Dijkstra.
[2] BFS.
[3] DFS.
[0] Volver al menú anterior.

Elige una opción: 2

Ingresa el nodo inicial: 0

Recorrido BFS:

Recorrido BFS desde 0 (Avenida_Adolfo_Lopez):
-> 0(Avenida_Adolfo_Lopez) -> 1(Glorieta_BenitoJuarez) -> 6(Isla_SanMarcos) -> 2(UAA) ->
5(Seguridad_Vial) -> 4(Costco) -> 3(Altaria)
```

Figura 8. Algoritmo BFS en ejecución.

2) Depth-First Search (DFS)

Implementado utilizando:

- Recursividad.
- Un arreglo de marcaje.
- Llamadas sucesivas para explorar vecinos.

Este algoritmo resulta útil para analizar conectividad profunda dentro de la red.

Resultados

A continuación, en la figura 9 se muestra el algoritmo DFS en ejecución.

```
=====
====> Recorridos del Grafo <====
[1] Dijkstra.
[2] BFS.
[3] DFS.
[0] Volver al menú anterior.

Elige una opción: 3

Ingresa el nodo inicial: 2

Recorrido DFS:

Recorrido DFS desde 2 (UAA):
-> 2(UAA) -> 4(Costco) -> 5(Seguridad_Vial) -> 6(Isla_SanMarcos) -> 3(Altaria) ->
1(Glorieta_BenitoJuarez)
```

Figura 9. Algoritmo DFS en ejecución.

3) Dijkstra para ruta mínima

El algoritmo empleado utiliza:

- Una **min-heap** propia.
- Un vector de distancias inicializado a infinito.
- Un vector de padres para reconstrucción de rutas.

El sistema toma como entrada un nodo origen y calcula las distancias mínimas hacia todos los demás nodos alcanzables. Además, se mide el tiempo de ejecución con **chrono** para evaluar el rendimiento del algoritmo.

Resultados

A continuación, en la figura 10 se muestra el algoritmo Dijkstra en ejecución.

```
=====
====> Recorridos del Grafo <====
[1] Dijkstra.
[2] BFS.
[3] DFS.
[0] Volver al menú anterior.

Elige una opción: 1

Ingresa el origen: 0

Ingresa el destino: 4

Camino mínimo 0 -> 4: 0 -> 1 -> 2 -> 4
Costo total: 10.8

Tiempo de ejecución de Dijkstra: 114.4 segundos
```

Figura 10. Algoritmo de Dijkstra en ejecución.

3.3 Tablas Hash y Almacenamiento Eficiente

El sistema incorpora dos estructuras hash independientes:

- **HashRed:** para almacenar nodos de la red leída desde CSV.
- **HashVehículos:** para administrar el registro de vehículos.

Ambas comparten las siguientes características:

1) Función hash

Se aplica una operación simple basada en el módulo del ID para determinar la posición del elemento en el arreglo base.

2) Manejo de colisiones por encadenamiento

Cada índice contiene una lista enlazada para almacenar múltiples elementos.

3) Operaciones principales

Cada tabla implementa métodos de:

- Inserción.
- Búsqueda por clave.
- Eliminación.
- Recorrido completo para depuración o verificación.

La inclusión de estas tablas permite búsquedas de complejidad y reduce significativamente el tiempo de acceso a los datos cargados.

3.4 Archivos CSV

Se desarrolló un módulo especializado para procesar archivos externos que contienen información de la red (nodos y aristas). Este módulo ejecuta las siguientes tareas:

1) Lectura desde CSV

El sistema permite cargar:

- Un conjunto de nodos con IDs y nombres.

- Una lista de aristas con pesos asociados.

El archivo se recorre línea por línea, separando campos y almacenándolos primero en la tabla HashRed para después reconstruir el grafo.

2) Construcción del grafo cargado

Una vez cargados los datos:

- Se reinicia el grafo dinámico.
- Se insertan los nodos encontrados en el hash.
- Se insertan las aristas contenidas en el vector leído.

Esto produce una reconstrucción fiel del grafo definido en el archivo CSV.

3) Modificación y guardado

Cuando se solicitan altas o bajas:

- Se abre el CSV seleccionado.
- Se modifican los registros según la operación requerida.
- Se guarda en el mismo archivo o en uno nuevo (modo "guardar como").

Este mecanismo facilita la creación de versiones evolucionadas de la red.

Resultados

En las figuras 11 y 11.1 se muestra la creación de archivos de red.

```
====> Nodos y Aristas (Manejo desde Archivos) <====
[1] Crear nueva Red.
[2] Alta de Nodos.
[3] Alta de Aristas.
[4] Baja de Nodos.
[5] Baja de Aristas.
[0] Volver al menú anterior.

Elige una opción: 1

Cuántos nodos deseas agregar: 3

Dame el nombre del nodo 0: Plaza_Principal
Dame el nombre del nodo 1: Avenida_Central
Dame el nombre del nodo 2: Universidad

Nodo origen: 0 (Plaza_Principal)
Deseas agregar una arista desde Plaza_Principal hacia Avenida_Central? [S/N]: S
Peso: 2.5
Deseas agregar una arista desde Plaza_Principal hacia Universidad? [S/N]: N
Nodo origen: 1 (Avenida_Central)
Deseas agregar una arista desde Avenida_Central hacia Plaza_Principal? [S/N]: S
Peso: 1
Deseas agregar una arista desde Avenida_Central hacia Universidad? [S/N]: S
Peso: 4.3
```

Figura 11. Creación de archivos de red, primera parte.

```
Deseas agregar una arista desde Universidad hacia Plaza_Principal? [S/N]: S
Peso: 4.8
Deseas agregar una arista desde Universidad hacia Avenida_Central? [S/N]: N
El archivo fue guardado correctamente.
```

Figura 11.1. confirmación de red creada, parte 2.

3.5 Archivos Binarios

Para llevar un control automático de las versiones de redes y vehículos, se implementaron dos archivos binarios:

- ContRed.dat
- ContVehiculos.dat

Ambos archivos almacenan enteros que representan el número de archivos generados.

1) Creación y verificación

Si el archivo no existe:

- Se genera automáticamente.
- Se inicializa el valor en 1.

2) Lectura y actualización

Cada vez que se crea una versión nueva:

- Se lee el último valor.
- Se incrementa.
- Se escribe al final del archivo.

Esto permite generar nombres como: red.csv, red2.csv, red3.csv, etc. Y de igual forma para vehículos.

Resultados

A continuación, en la figura 12 se muestra cómo se carga la red con éxito.

```
====> Red, Nodos y Tablas Hash <====
[1] Cargar Red.
[2] Opciones de Red usando Archivo.
[3] Opciones de Red usando Grafo.
[0] Volver al menú principal.

Elige una opción: 1

Archivos de Redes de Nodos:

[1] red.csv
[2] red2.csv
[3] red3.csv

Qué archivo desea cargar: 3

Red cargada correctamente desde red3.csv
```

Figura 12. Red cargada con nombre.

3.6 Módulo de Gestión de Vehículos

El sistema incorpora un módulo dedicado al manejo de vehículos, el cual permite administrar la información almacenada tanto en archivos CSV como en estructuras internas (Hash y Grafo). Este módulo se organiza mediante tres funciones principales:

1) Menú general de vehículos

Esta función actúa como menú principal de operaciones con vehículos. Desde aquí el usuario puede:

- Cargar un archivo CSV de vehículos
- Se solicita el nombre del archivo.
- Se reinicia la tabla hash de vehículos.
- Se limpia el vector vehics.
- Se carga el archivo usando csvVehiculos, llenando tanto el hash como la lista de vehículos.
- Mostrar los vehículos cargados actualmente
- Recorre el vector vehics y despliega toda la información disponible.
- Accede a opciones basadas en archivos (crear, buscar, eliminar o agregar vehiculos directamente en CSV).
- Accede a opciones basadas en grafo (simular movimientos con Dijkstra, altas/bajas desde hash, búsquedas, etc.).

Este menú integra todos los componentes del proyecto (archivos, hash y grafo) en un punto central de interacción.

Resultados

```
=====
====> Opciones de Vehículos (Desde archivo) <====
[1] Alta de Vehículo.
[2] Buscar Vehículo.
[3] Baja de Vehículo.
[4] Crear nuevo archivo de vehículos.
[0] Regresar al menú anterior.

Elige una opción:
[0] Regresar al menu anterior.

Elige una opción:
```

En la figura 13 se muestra el menú general de vehículos.

Figura 13. Menú general de vehículos.

A continuación, en las figuras 14 y 14.1 se muestra la carga de vehículos desde archivo.

```
=====
====> Opciones de Vehículos <====
[1] Cargar archivo de vehículos.
[2] Mostrar vehículos cargados actualmente.
[3] Opciones de Vehículos (Con archivos).
[4] Opciones de Vehículos (Con Grafo).
[0] Regresar al menú anterior.

Elige una opción: 1

Archivos de Vehículos:

[1] vehiculos.csv
[2] vehiculos2.csv

Elija un archivo para trabajar con él: 2
=====
```

Figura 14. Carga de archivo de vehículos.

```
=====
====> Opciones de Vehículos <====
[1] Cargar archivo de vehículos.
[2] Mostrar vehículos cargados actualmente.
[3] Opciones de Vehículos (Con archivos).
[4] Opciones de Vehículos (Con Grafo).
[0] Regresar al menú anterior.

Elige una opción: 2

Los vehículos cargados actualmente son:

Id: 0 | Placa: TFS456 | Tipo: Transporte | Origen: 1 | Destino: 5 | Hora de entrada: 07:36
Id: 1 | Placa: WOF152 | Tipo: Particular | Origen: 2 | Destino: 6 | Hora de entrada: 12:25
Id: 2 | Placa: MXN364 | Tipo: Transporte | Origen: 0 | Destino: 3 | Hora de entrada: 09:52
Id: 3 | Placa: VSV117 | Tipo: Particular | Origen: 6 | Destino: 3 | Hora de entrada: 20:30
Id: 4 | Placa: CAZ974 | Tipo: Particular | Origen: 4 | Destino: 0 | Hora de entrada: 17:15
=====
```

Figura 14.1. carga actual de vehículos.

2) Manejo directo desde archivos CSV

Esta sección permite modificar la información **sin usar estructuras dinámicas**, actuando directamente sobre los archivos:

- **Alta de vehículo:** Agrega un nuevo registro al archivo indicado.
- **Búsqueda de vehículo:** Lee el archivo y localiza un vehículo por ID o placa.
- **Baja de vehículo:** Crea una versión filtrada del archivo donde se elimina el registro solicitado.
- **Crear un nuevo archivo de vehículos:** Permite generar un archivo CSV completamente nuevo.

Este módulo garantiza la persistencia de datos aun cuando el grafo o la tabla hash no se estén utilizando.

Resultados

En la figura 15 se muestra el menú general desde archivos.

```
=====
====> Opciones de Vehículos (Desde archivo) <====
[1] Alta de Vehículo.
[2] Buscar Vehículo.
[3] Baja de Vehículo.
[4] Crear nuevo archivo de vehículos.
[0] Regresar al menú anterior.

Elige una opción:
```

Figura 15. Operaciones con vehículos desde archivo.

A continuación, se muestra en la figura 16 y 16. 1, la creación de un vehículo desde archivo con todas sus características.

```
=====
====> Opciones de Vehículos (Desde archivo) <====
[1] Alta de Vehículo.
[2] Buscar Vehículo.
[3] Baja de Vehículo.
[4] Crear nuevo archivo de vehículos.
[0] Regresar al menú anterior.

Elige una opción: 4

Cuántos vehículos desea ingresar: 3

-- Vehículo 1 --
Ingresa la placa del vehículo (Formato AAA000): HGF777

[1] Particular
[2] Transporte
Escoja el tipo de vehículo: 1

Ingresa el nodo de origen del vehículo: 3

Ingresa el nodo de destino del vehículo: 5

Ingresa la hora de entrada del vehículo: 13:41

-- Vehículo 2 --
Ingresa la placa del vehículo (Formato AAA000): BVC379

[1] Particular
[2] Transporte
Escoja el tipo de vehículo: 2

Ingresa el nodo de origen del vehículo: 2

Ingresa el nodo de destino del vehículo: 3

Ingresa la hora de entrada del vehículo: 10:52
```

Figura 16. Creación de un vehículo, primera parte.

```
-- Vehículo 3 --
Ingresa la placa del vehículo (Formato AAA000): KOL756

[1] Particular
[2] Transporte
Escoja el tipo de vehículo: 1

Ingresa el nodo de origen del vehículo: 5

Ingresa el nodo de destino del vehículo: 1

Ingresa la hora de entrada del vehículo: 12:36

El archivo fue creado correctamente
=====
```

Figura 16.1. Creación de vehículo, segunda parte.

3) Gestión de vehículos con grafo y Dijkstra

Este menú se enfoca en el uso del grafo y de la tabla hash para manejar vehículos ya cargados. Sus funciones principales son:

Aplicar Dijkstra a un vehículo

- Se muestran todos los vehículos cargados.
- El usuario ingresa el ID del vehículo.
- Se obtiene el puntero al registro mediante la tabla hash.
- Se ejecuta **Dijkstra** desde el nodo de origen del vehículo.
- Se almacena el vector de distancias y el arreglo de padres.

- Se mide el tiempo de ejecución del algoritmo utilizando std::chrono.
- Se imprime el camino desde el origen al destino usando imprimirCamino().
- Finalmente, se reporta el tiempo total del cálculo.

Alta, búsqueda y baja de vehículos usando hash

- **Alta:** Se genera un nuevo ID, se llena el objeto Vehículo y se agrega tanto al hash como al vector.
- **Búsqueda:** Se consulta directamente en la tabla hash, evitando recorrer archivos o listas.
- **Baja:** Se elimina el vehículo de la tabla hash de forma inmediata.

Este módulo demuestra el uso conjunto de:

- estructuras dinámicas (hash),
- estructuras de grafos,
- algoritmos de caminos mínimos,
- y medición de rendimiento.

Resultados

En la figura 17 se muestra el menú de opciones desde grafo para operaciones con vehículos.

```
=====
====> Opciones de Vehículos (Desde grafo) <====
[1] Aplicar Dijkstra a un Vehículo.
[2] Alta de Vehículo.
[3] Buscar Vehículo.
[4] Baja de Vehículo.
[0] Regresar al menú anterior.

Elige una opción:
```

Figura 17. Operaciones con vehículos desde grafo.

En la figura 18 se muestra la aplicación de Dijkstra a un vehículo.

```
=====
====> Opciones de Vehículos (Desde grafo) <====
[1] Aplicar Dijkstra a un Vehículo.
[2] Alta de Vehículo.
[3] Buscar Vehículo.
[4] Baja de Vehículo.
[0] Regresar al menú anterior.

Elige una opción: 1

Los vehículos contenidos en el archivo son:
Id: 0 | Placa: TFS456 | Tipo: Transporte | Origen: 1 | Destino: 5 | Hora de entrada: 07:36
Id: 1 | Placa: KOF152 | Tipo: Particular | Origen: 2 | Destino: 6 | Hora de entrada: 12:25
Id: 2 | Placa: RYM364 | Tipo: Transporte | Origen: 0 | Destino: 3 | Hora de entrada: 09:52
Id: 3 | Placa: VSV117 | Tipo: Particular | Origen: 6 | Destino: 3 | Hora de entrada: 20:30
Id: 4 | Placa: CAZ974 | Tipo: Particular | Origen: 4 | Destino: 0 | Hora de entrada: 17:15

Ingresa el id del vehículo que quieres simular su movimiento: 3

Camino mínimo 6 -> 3: 6 -> 2 -> 4 -> 3
Costo total: 13.9

Tiempo de movimiento del automóvil: 18.2 segundos

=====
```

Figura 18. Aplica Dijkstra a un vehículo.

En la figura 19 se muestra la búsqueda de un vehículo.

```
=====
====> Opciones de Vehículos (Desde archivo) <====
[1] Alta de Vehículo.
[2] Buscar Vehículo.
[3] Baja de Vehículo.
[4] Crear nuevo archivo de vehículos.
[0] Regresar al menú anterior.

Elige una opción: 2

Archivos de Vehículos:
[1] vehiculos.csv
[2] vehiculos2.csv

Elija un archivo para trabajar con él: 1

[1] Buscar por Id
[2] Buscar por placa

Cómo quieres buscar el vehículo: 1

Ingresa el Id el vehículo a buscar: 4

-- Vehículo --
Id: 4 | Placa: CAZ974 | Tipo: Particular | Origen: 4 | Destino: 0 | Hora de entrada: 17:15

=====
```

Figura 19. Búsqueda de vehículo.

3.7 Interfaz del Usuario y Módulo Principal

El archivo main actúa como núcleo de control e interacción. El sistema se organiza mediante un menú que agrupa las funciones:

1) Menú principal

Resultados

En la figura 20 se muestra el menú de Red, Nodos y Hash.

```
=====
====> MENÚ PRINCIPAL <====
[1] Opciones de Red, Nodos y tablas Hash.
[2] Matriz y Lista del Grafo.
[3] Recorridos del grafo.
[4] Opciones de vehículos.
[0] Salir del programa.

Elige una opción: |
```

Figura 20. Menú principal con opciones.

Finalmente, se configura la consola en UTF-8, se manejan validaciones de entrada y se evita la ejecución de algoritmos sobre nodos inexistentes.

IV. DISCUSIÓN

Los resultados obtenidos muestran que el sistema cumple de manera efectiva con la administración dinámica de redes de nodos, la representación de grafos y la ejecución de algoritmos de recorrido. La carga desde archivos CSV y el manejo del archivo contador (**ContRed.dat**) demostraron ser mecanismos confiables para gestionar diferentes versiones de la red, facilitando la persistencia y la recuperación de información sin afectar la integridad del grafo.

Las operaciones de alta y baja de nodos y aristas se reflejaron correctamente tanto en la matriz de adyacencia como en la lista de adyacencia, evidenciando coherencia interna en la estructura del grafo. Con la incorporación de validaciones adicionales, se redujo la posibilidad de errores por entradas inválidas o referencias a nodos inexistentes, mejorando la robustez general del sistema.

En cuanto a los algoritmos, Dijkstra arrojó tiempos de ejecución coherentes con el tamaño del grafo utilizado, mientras que BFS y DFS generaron recorridos válidos, lo cual confirma que la implementación se apega correctamente al funcionamiento esperado de dichas técnicas. Esto muestra que el grafo dinámico mantiene una estructura estable incluso después de múltiples modificaciones.

El módulo de vehículos demostró un funcionamiento adecuado en la gestión del archivo contador y en las primeras pruebas de persistencia.

En conjunto, el sistema presenta una arquitectura modular y funcional, con buena interacción entre sus componentes principales.

V. CONCLUSIONES

El proyecto logró implementarse de manera íntegra y satisfactoria, cumpliendo con todos los requisitos funcionales relacionados con la gestión, análisis y manipulación dinámica de una red representada mediante un grafo. La arquitectura final integra correctamente múltiples módulos: carga automática de topologías desde archivos CSV, administración completa de nodos y aristas tanto desde archivo como desde estructuras del programa, utilización de tablas hash para búsquedas eficientes, y ejecución de algoritmos de recorrido fundamentales como BFS, DFS y Dijkstra.

El sistema demostró ser estable y consistente en todas las pruebas realizadas. La funcionalidad de iniciar por defecto permitió cargar redes complejas sin errores, reconstruyendo nodos y aristas en memoria mediante estructuras dinámicas. Las opciones de alta, baja y modificación mostraron un manejo adecuado de memoria, evitando duplicidades y garantizando la integridad del grafo. Asimismo, la visualización mediante matriz y lista de adyacencia confirmó la correcta representación interna de la red en todo momento.

El módulo de recorridos produjo resultados precisos para las rutas mínimas (Dijkstra) y recorridos por niveles y profundidad (BFS y DFS), incorporando además mediciones de tiempo que permitieron evaluar su rendimiento en redes de distinto tamaño. Por otro lado, la sección independiente de vehículos añadió una capa adicional de manejo de datos que se integró de forma fluida en la estructura general del proyecto.

En conjunto, el sistema final constituye una herramienta flexible y completamente funcional. La combinación de estructuras dinámicas, archivos externos, tablas hash y algoritmos de grafo confirma el cumplimiento pleno de los objetivos planteados.

REFERENCIAS

- [1] Baeldung. (2024, 31 de julio). DFS vs BFS vs Dijkstra. <https://www.baeldung.com/cs/dfs-vs-bfs-vs-dijkstra>
- [2] **Material del curso proporcionado por el profesor**, notas de clase, ejemplos de código y documentos de apoyo del curso de *Estructuras de Datos II*, 2025.

Apéndice A: link de GitHub https://github.com/vlady-sv/Sistema.TraficoUrbano_EDII/tree/main

Apéndice B: diagrama de flujo del proyecto https://drive.google.com/drive/folders/1n7OjgA-hduB50SLvb_4XhH9b6Hzz9JXD?usp=sharing